



香港中文大學
The Chinese University of Hong Kong



Microsoft

Saving Lost Intents in Network Configurations

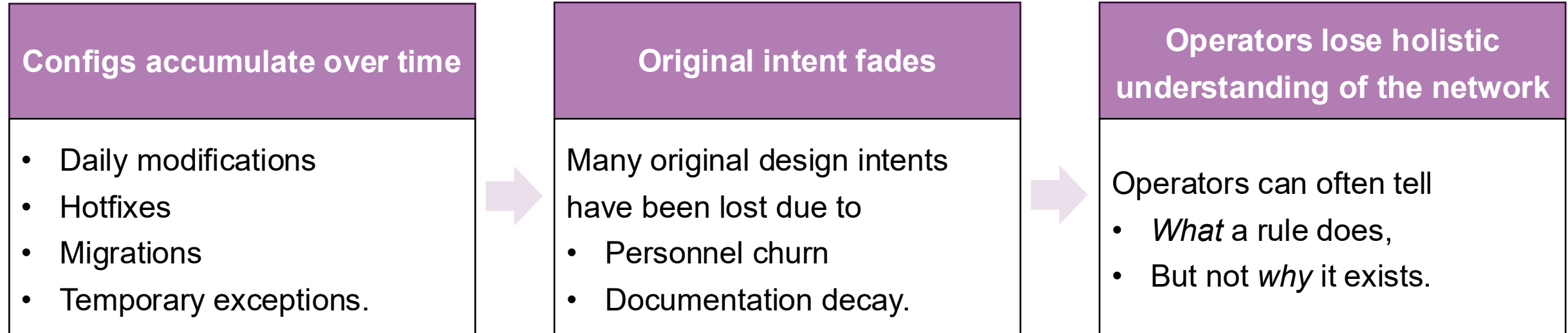
Jianqiang Li¹, Wenlong Ding¹, Zhixiong Niu², Yongqiang Xiong²,
Hong Xu¹

¹The Chinese University of Hong Kong, ²Microsoft Research Asia

Background: NetOps in Brownfield Environments



- Why Intent Recovery?



We need to recover intent across the entire network configs over time.

Background: NetOps in Brownfield Environments



- Why Intent Recovery? (An Example)



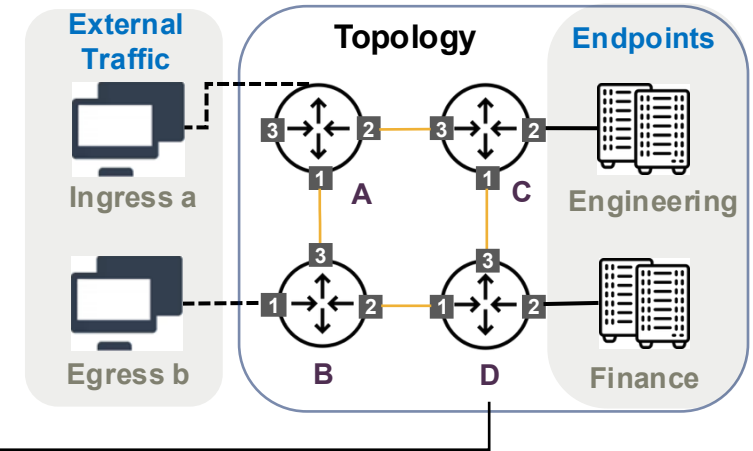
NetOps

Operator Audit: I see there is a filtering rule on router D. Can I remove it now? What's the original business intent?

D.cfg (excerpt):

```
ip access-list extended FINANCE-MAINT
deny tcp 10.1.1.0/24 host 10.2.2.10 eq 5432
permit ip any any
```

```
interface Gi0/2
ip access-group FINANCE-MAINT in
```



Unfortunately, there are multiple reasonable explanations:

Permanent Engineering-to-Finance isolation

Temporary annual-maintenance block

Stale exception after migration

The operator can only do nothing for safety reasons due to the lack of clear intent in the existing network.

Intent Recovery:

- **Input:** A set of observed network properties within a user-specified scope
- **Output:** A set of high-level *intended policies* that best explain these properties
- **Subject to:** Consistency with known context

Intent Recovery is the **inverse** problem of *Intent Translation*.

The inverse problem is **ill-posed**, i.e., the mapping from the *configuration space* to the *intent space* is one-to-many.

Intent Recovery is fundamentally distinct from ***Verification*** or ***Specification Mining***

- *Verification* asks **whether** a property holds.
- *Specification Mining* asks **what** properties hold.
- *Intent Recovery* asks **why** a property holds.

Motivation: Why Hard and Important



Context decay

- **Semantics**: business roles become flat IPs.
- **Provenance**: one policy fragments across config lines/devices.
- **History**: later updates hide earlier design rationale.

Brownfield IBN is untenable

- Current IBN often assumes a complete intent set.
- Brownfield networks only have legacy configs.
- Recovered intents provide constraints for safe future changes.

Motivation: Existing Approaches Fail



1 Manuals & Commit Logs

Abstraction-level mismatch: From 100+ PRs in a production cloud:

- **69.9%** implementation-centric
- **24.3%** process-oriented
- **Only 3.9%** state the intent

2 Formal Analysis (Batfish, Config2Spec, NetMiner)

Answers “**What Is**”, not “**Why It Is**.”

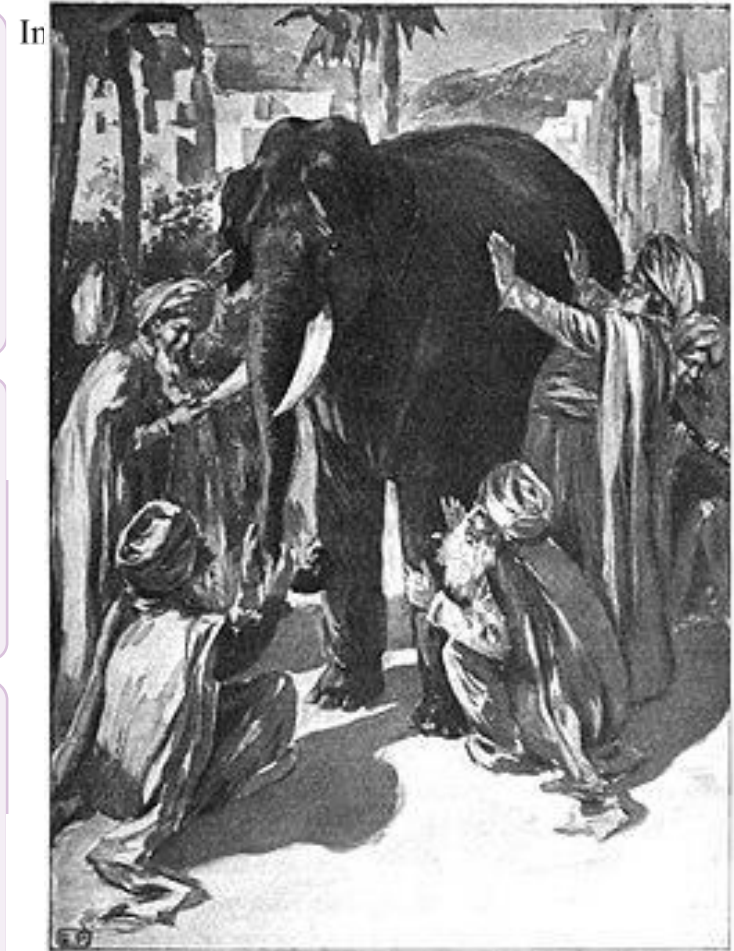
- Without context, formal mining methods optimizes for compactness and conflates independent policies.

3 LLM Summarization (Copilot-style)

On the same PRs:

- **93.9%** only paraphrase the diff
- **0%** infer the policy

An input-context limit, not a model-quality limit.



Each source sees only an **incomplete projection** of the context; none alone can recover intent.

Key Insight: Regularization via Context



Observation: The *snapshot* alone is underdetermined, but the original design leaves *residual traces* in surrounding artifacts.



Insight: Use context as regularization.

Q1. What are the useful residual traces and how to mine the context?

“Why 10.1.1.0/24 cannot reach 10.2.2.10:5432? What’s the original intent?”

Semantic context (Business entity mapping)

- 10.1.1.0/24→Engineering workstations
- 10.2.2.10→Finance DB ∈ Finance Dept
- 5432→PostgreSQL

Provenance context (Local implementation trace)

- Traceback: the deny rule on D.cfg
- No other nearby rules that share similar fields

Temporal context (Git history of the implementation)

- Added within a maintenance batch
- Inserted after (in time) an older permit policy
- Left behind after a migration

Key Insight: Regularization via Context



Observation: The *snapshot* alone is underdetermined, but the original design leaves *residual traces* in surrounding artifacts.



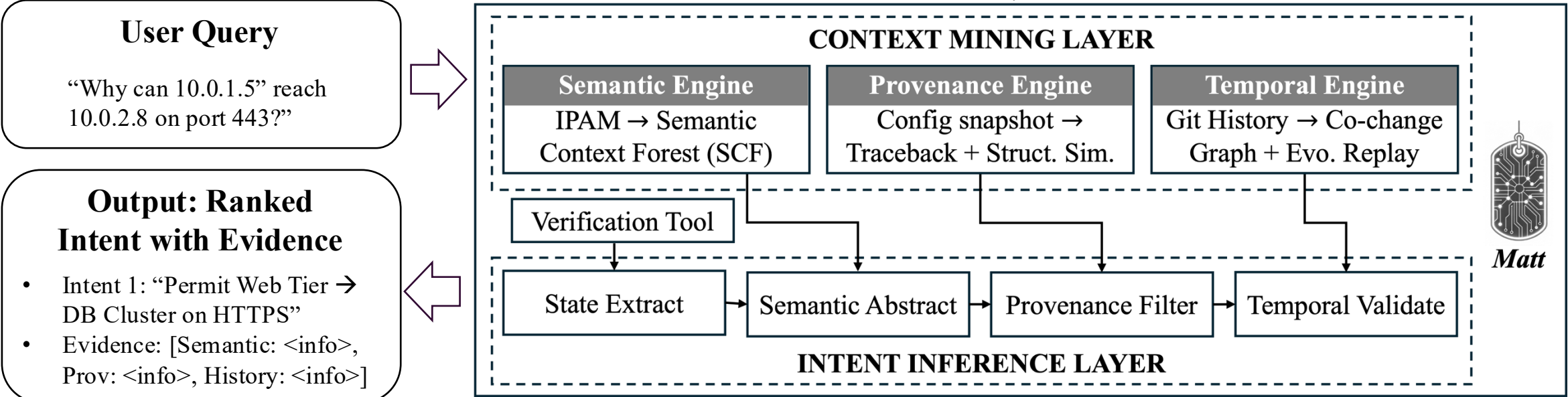
Insight: Use context as regularization.

Q2. How to use the context to infer the original intents?

$$\mathcal{I}^* = \arg \max_{\mathcal{I}} \underbrace{P(S_Q | \mathcal{I})}_{\text{Likelihood}} \cdot \underbrace{P(\mathcal{I} | C_{xt})}_{\text{Prior}}$$

Likelihood: Does the proposed intent explain the observed network state?

Prior: Does the intent align with semantic, structural, and historical evidence?



Bounded task: explain a queried state, not reconstruct all policies.

Hard grounding: verification identifies behavior and contributing lines.

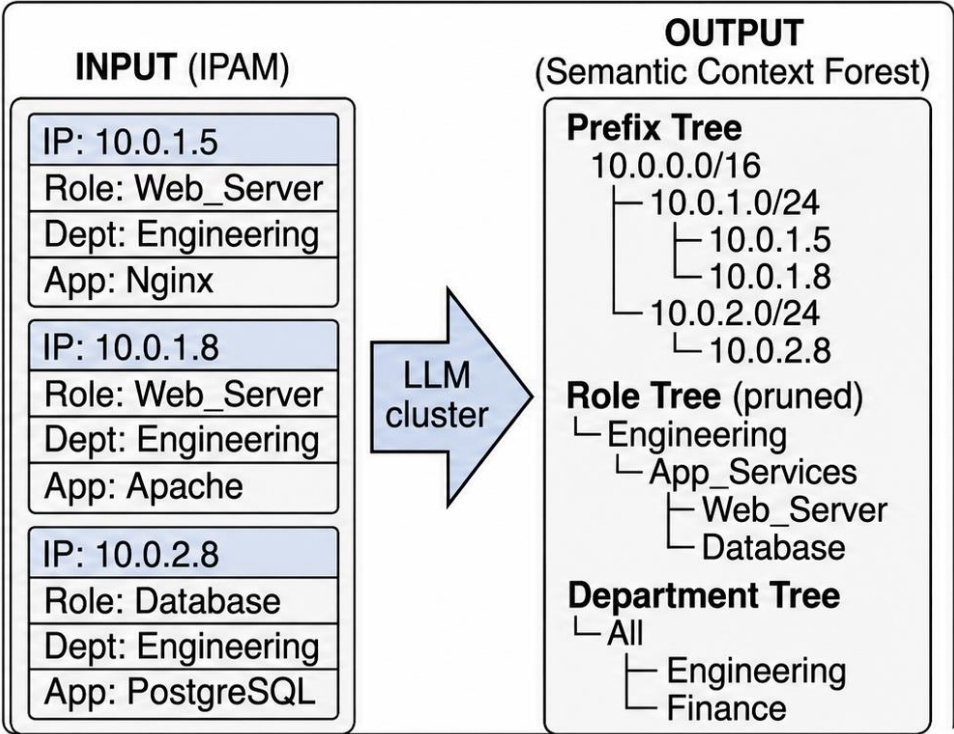
Soft evidence: semantics / provenance / history act as priors.

Progressive refinement: generate broad semantic candidates, then prune by structural and temporal support.

Design 1: Semantic Abstraction



- **Challenge:** one IP sits at many levels of many hierarchies.



Panel (a): IPAM metadata → Semantic Context Forest

Query: ‘Why can 10.0.1.5, 10.0.1.8 reach 10.0.2.8?’

Candidate abstractions for source IP set $S = \{10.0.1.5, 10.0.1.8\}$:

Keep every abstraction A whose lifted state remains consistent with the query.

Abstraction A	Coverage	State holds?	Keep?
All Hosts	512 IPs	No	✗
Engineering	128 IPs	No	✗
App_Services	32 IPs	No	✗
Web_Server	16 IPs	Yes	✓
exact IPs	2 IPs	Yes	✓

Panel (b): Likelihood-based candidate selection

Semantic Context Forest (SCF): prefix and metadata hierarchies built from the IPAM table.

Candidate generation: enumerate abstractions across levels and dimensions, keeping those whose lifted state still holds.

Design 2: Provenance Filtering



- Challenge:** business-similar flows can be unrelated in the config.

Reverse tracing:

$$\mathcal{C}_f = \{(d_i, l_i)\}$$

For each flow $f \in S_Q$, use verification to collect config lines that contribute to the queried flow/behavior.

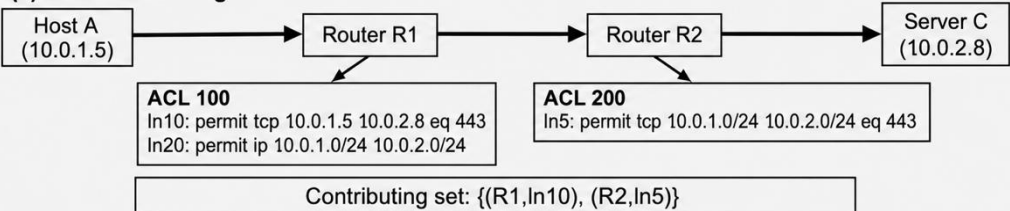


Structural affinity:

$$\text{Aff}(l_i, l_j) = \alpha \mathbf{1}_{ij}^{\text{block}} + (1 - \alpha) J(M_i, M_j)$$

- $\mathbf{1}_{ij}^{\text{block}}$: same config block?
- $J(M_i, M_j)$: Jaccard *similarity* of match fields.
- Group lines if $\text{Aff}(l_i, l_j) \geq \tau_p$.

Panel (a): Reverse Tracing



Panel (b): Structural Affinity and Regrouping

Affinity between (R1,ln10) and (R2,ln5):	$1_{\text{block}} = 0$ (different devices)	J = 1/3 = 0.33
J(M1, M2): {src:10.0.1.5, dst:10.0.2.8, port:443} vs {src:10.0.1.0/24, dst:10.0.2.0/24, port:443}		Aff = 0.4 × 0.33 = 0.13
Affinity between (R1,ln10) and (R1,ln20):	$1_{\text{block}} = 1$ (same ACL)	J = 0
J(M1, M2): {src:10.0.1.5, dst:10.0.2.8, port:443} vs {src:10.0.1.0/24, dst:10.0.2.0/24}		Aff = 0.6 × 1 + 0.4 × 0 = 0.6
Regrouping Result (threshold $\tau_p = 0.5$):		
Group {(R1,ln10), (R1,ln20)}		(same ACL block, high affinity)
(R2,ln5)		(kept separate; cross-device affinity is low)

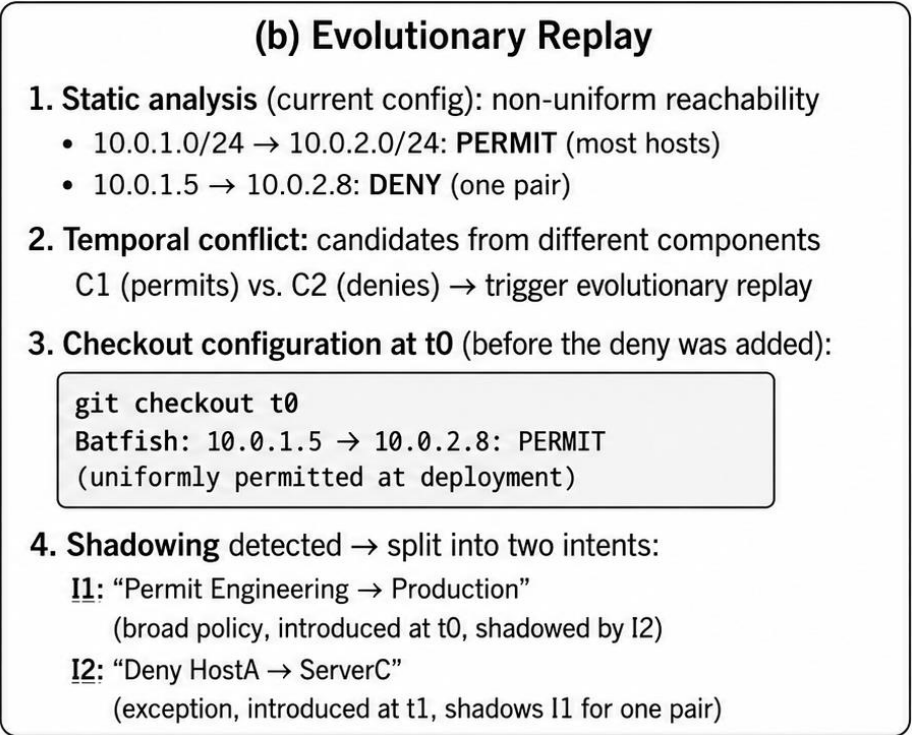
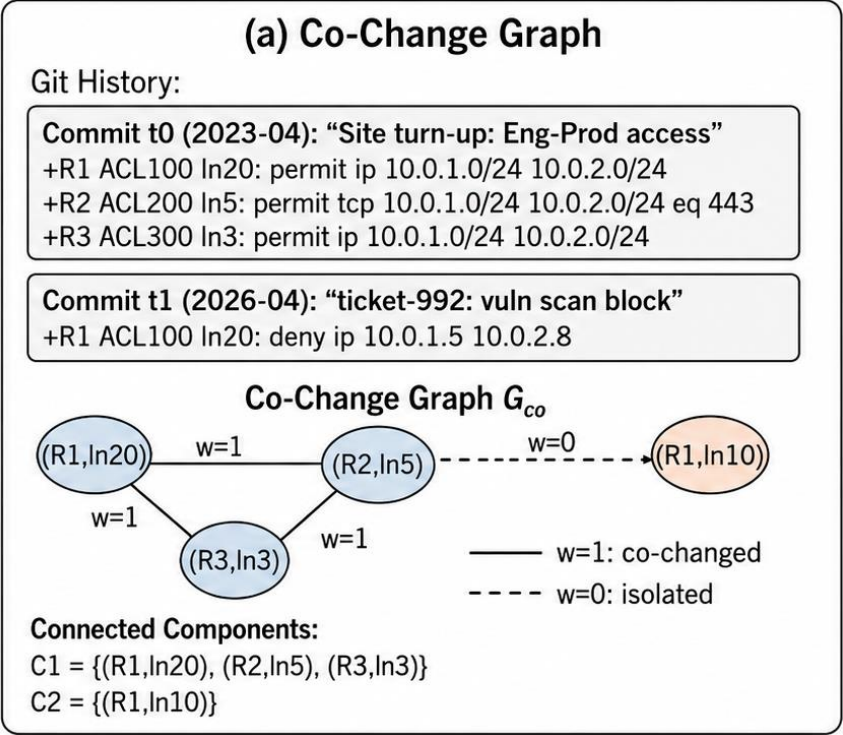
Reverse tracing: find the lines that produce the queried state.

Affinity & regrouping: score block locality and match-field similarity, then split candidates that lack support.

Design 3: Temporal Validation



- Challenge:** a permanent policy and a temporary exception look identical in current configs.



Co-change Identification: lines edited together form one deployment lifecycle.

Evolutionary Replay: when a later narrow rule shadows a broad one, replay pre-override snapshot.

Preliminary Results



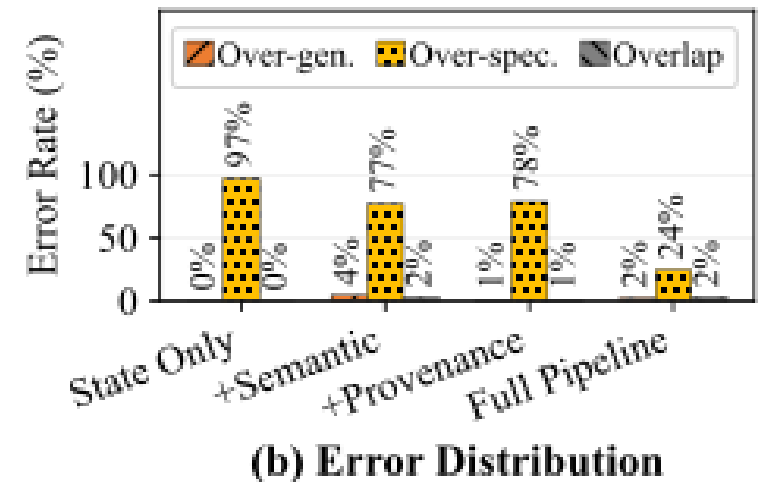
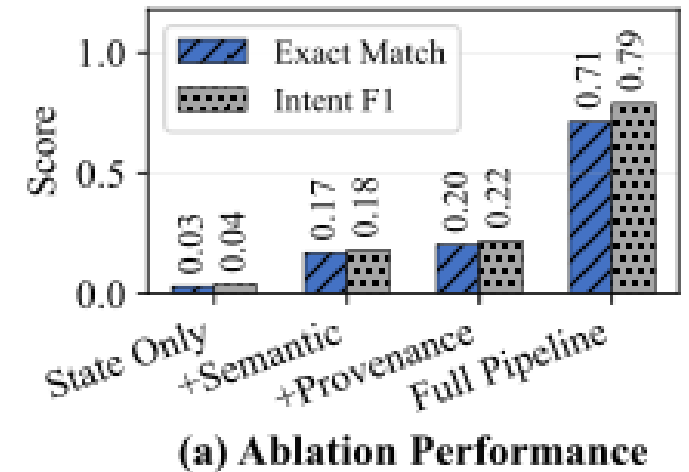
We evaluate preliminary results on a synthetic dataset.

Finding 1: Verification alone cannot recover intent

- *State Only*: **0.03 EM / 0.04 F1**
 - 97% of errors are over-specialization
 - Explains behavior at IP/rule level, never at intent level

Finding 2: The three context dimensions are complementary

- *Semantics*: lifts IPs into business roles
- *Provenance*: removes over-merge (4% → 1%)
- *History*: resolves the rest ambiguity
- *Together*: **0.71 EM / 0.79 F1**, error 80% → 29%



Summary & Future Work



Core problem: Production configs are often append-only

The **why** is gone, and recovering it is ill-posed.

Why it is hard: Every source is partial

- Commit logs (but only 3.9% accuracy on intents).
- Verification says *what*, but not *why*.
- LLM (but only 0% accuracy on intents).

Our insight: Regularize with the observed design trace

Semantics + Provenance + History (fused as MAP inference).

Where this goes

- A joint solver over all three dimensions.
- Deployment on a production cloud network.
- Unlocking **brownfield IBN**: safe automation for networks we already run.